

CS 450 – Numerical Analysis

Chapter 9: Initial Value Problems for Ordinary Differential Equations [†]

Prof. Michael T. Heath

Department of Computer Science
University of Illinois at Urbana-Champaign
heath@illinois.edu

January 28, 2019

[†]Lecture slides based on the textbook *Scientific Computing: An Introductory Survey* by Michael T. Heath, copyright © 2018 by the Society for Industrial and Applied Mathematics. <http://www.siam.org/books/cl80>

Ordinary Differential Equations

Differential Equations

- ▶ Differential equations involve derivatives of unknown solution function

$$\frac{d^n}{dt^n} f(t) = 0$$

- ▶ **Ordinary differential equation** (ODE): all derivatives are with respect to single independent variable, often representing time

- ▶ Solution of differential equation is continuous function in infinite-dimensional space of functions

Approximate

- ▶ **Numerical** solution of differential equations is based on finite-dimensional approximation

- ▶ Differential equation is replaced by algebraic equation whose solution approximates that of given differential equation

analytical
closed-form
exact

$$\begin{bmatrix} f_0 \\ f_1 \\ \vdots \\ f_N \end{bmatrix} \in \mathbb{R}^{N+1}$$

$$\tilde{f} \in \mathbb{R}^{N+1}$$

Order of ODE



- ▶ **Order** determined by highest-order derivative of solution function appearing in ODE
- ▶ ODE with higher-order derivatives can be transformed into equivalent first-order system
- ▶ We will discuss numerical solution methods only for first-order ODEs
- ▶ Most ODE software is designed to solve only first-order equations

Higher-Order ODEs

- For k -th order ODE

$$y: [a, +\infty) \rightarrow \mathbb{R}$$

$$y(t) \rightarrow y(t)$$

$$y^{(k)}(t) = f(t, y, y', \dots, y^{(k-1)})$$

define k new unknown functions

$y \in C^k$

$$u_1(t) = y(t), \quad u_2(t) = y'(t), \quad \dots, \quad u_k(t) = y^{(k-1)}(t)$$

$$u_k'(t) = y^{(k)}(t) = f(t, y, y', \dots, y^{(k-1)})$$

- Then original ODE is equivalent to first-order system

$$\begin{bmatrix} u_1(t) \\ \vdots \\ u_k(t) \end{bmatrix} \in \mathbb{R}^k$$

$$= Y$$

y

$$\begin{bmatrix} u_1'(t) \\ u_2'(t) \\ \vdots \\ u_{k-1}'(t) \\ u_k'(t) \end{bmatrix} = \begin{bmatrix} u_2(t) \\ u_3(t) \\ \vdots \\ u_k(t) \\ f(t, u_1, u_2, \dots, u_k) \end{bmatrix}$$

$$Y' = f(t, Y)$$

Example: Newton's Second Law

- ▶ Newton's Second Law of Motion, $F = ma$, is second-order ODE, since acceleration a is second derivative of position coordinate, which we denote by y

- ▶ Thus, ODE has form

$$y'' = F/m$$

where F and m are force and mass, respectively

$x(t)$ - position
 $\dot{x}(t)$ - velocity
 $\ddot{x}(t)$ - acceleration

- ▶ Defining $u_1 = y$ and $u_2 = y'$ yields equivalent system of two first-order ODEs

$$Y = \begin{bmatrix} y \\ y' \end{bmatrix}$$

$$Y' = \begin{bmatrix} u_1' \\ u_2' \end{bmatrix} = \begin{bmatrix} u_2 \\ F/m \end{bmatrix}$$

$$Y' = \begin{bmatrix} y' \\ y'' \end{bmatrix}$$

- ▶ We can now use methods for first-order equations to solve this system

$$Y' = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} + \begin{bmatrix} 0 \\ F/m \end{bmatrix}$$

- ▶ First component of solution u_1 is solution y of original second-order equation

$$Y' = AY + B = f(Y)$$

- ▶ Second component of solution u_2 is velocity y'

Ordinary Differential Equations

- ▶ General first-order system of ODEs has form

$$\mathbf{y}'(t) = \mathbf{f}(t, \mathbf{y})$$

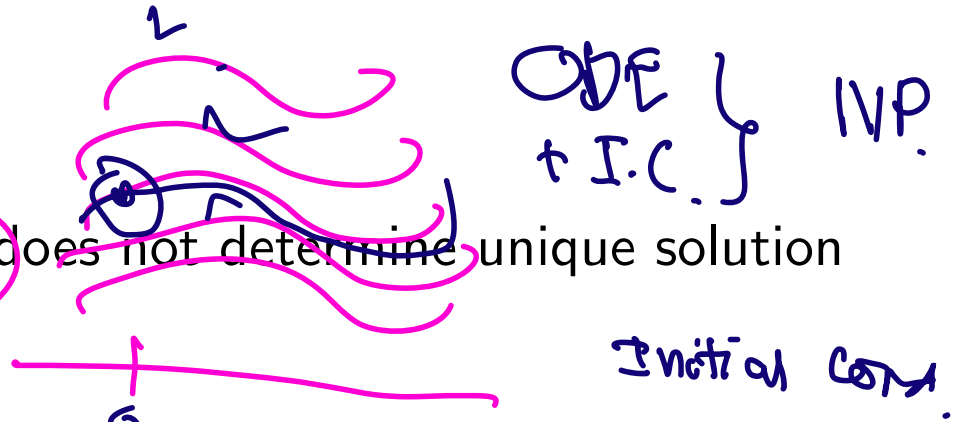
where $\mathbf{y}: \mathbb{R} \rightarrow \mathbb{R}^n$, $\mathbf{f}: \mathbb{R}^{n+1} \rightarrow \mathbb{R}^n$, and $\mathbf{y}' = d\mathbf{y}/dt$ denotes derivative with respect to t ,

$$\mathbf{y}' = \begin{bmatrix} y_1'(t) \\ y_2'(t) \\ \vdots \\ y_n'(t) \end{bmatrix} = \begin{bmatrix} dy_1(t)/dt \\ dy_2(t)/dt \\ \vdots \\ dy_n(t)/dt \end{bmatrix}$$

- ▶ Function $\mathbf{f}$ is given and we wish to determine unknown function $\mathbf{y}$ satisfying ODE
- ▶ For simplicity, we will often consider special case of single scalar ODE, $n = 1$

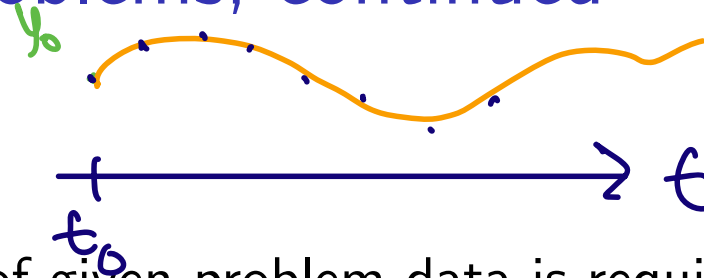
Initial Value Problems

- By itself, ODE $y' = f(t, y)$ does not determine unique solution function



- This is because ODE merely specifies *slope* $y'(t)$ of solution function at each point, but not actual value $y(t)$ at any point
- If $y(t)$ is solution and c is any constant, then $y(t) + c$ is also a solution because $d(y(t) + c)/dt = y'(t) + 0 = y'(t)$
- Infinite family of functions satisfies ODE, in general, provided f is sufficiently smooth
- To single out particular solution, value y_0 of solution function must be specified at some point t_0

Initial Value Problems, continued



↓
+C

- ▶ Thus, part of given problem data is requirement that $\mathbf{y}(t_0) = \mathbf{y}_0$ which determines unique solution to ODE
- ▶ Because of interpretation of independent variable t as time, we think of t_0 as initial time and $\mathbf{y}_0$ as initial value
- ▶ Hence, this is termed *initial value problem*, or *IVP*
- ▶ ODE governs evolution of system in time from its initial state $\mathbf{y}_0$ at time t_0 onward, and we seek function $\mathbf{y}(t)$ that describes state of system as function of time

Example: Initial Value Problem

$$\int \frac{y'}{y} dy = \ln(y) + C.$$

- Consider scalar ODE

$$y' = y$$

$$y' = (\lambda)y$$

$$y(t) = c e^{\lambda t}$$

- Family of solutions is given by $y(t) = c e^t$, where c is any real constant

- Imposing initial condition $y(t_0) = y_0$ singles out unique particular solution

- For this example, if $t_0 = 0$, then $c = y_0$, which means that solution

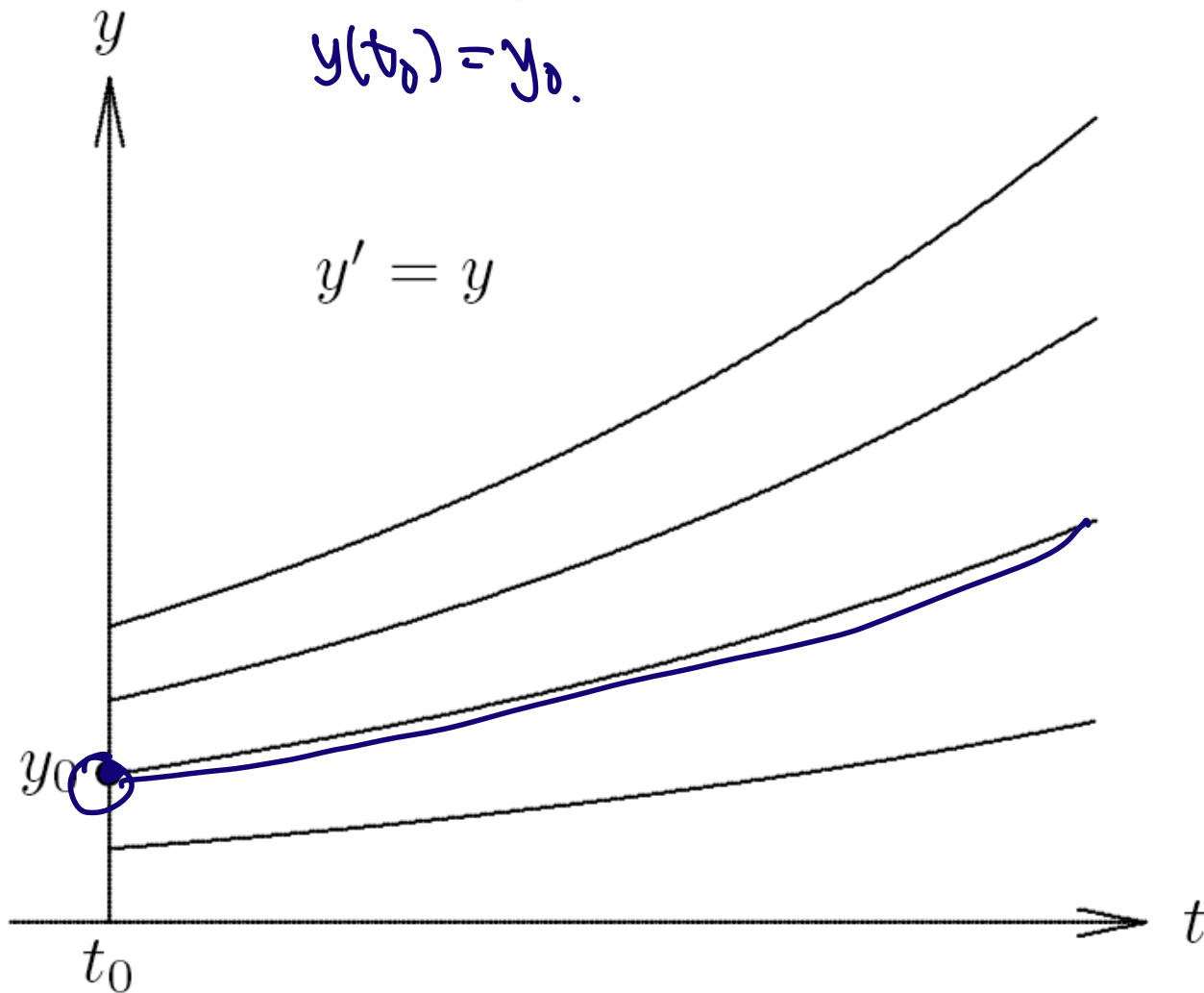
$$y(t) = y_0 e^t$$

Example: Initial Value Problem

Family of solutions for ODE $y' = y$

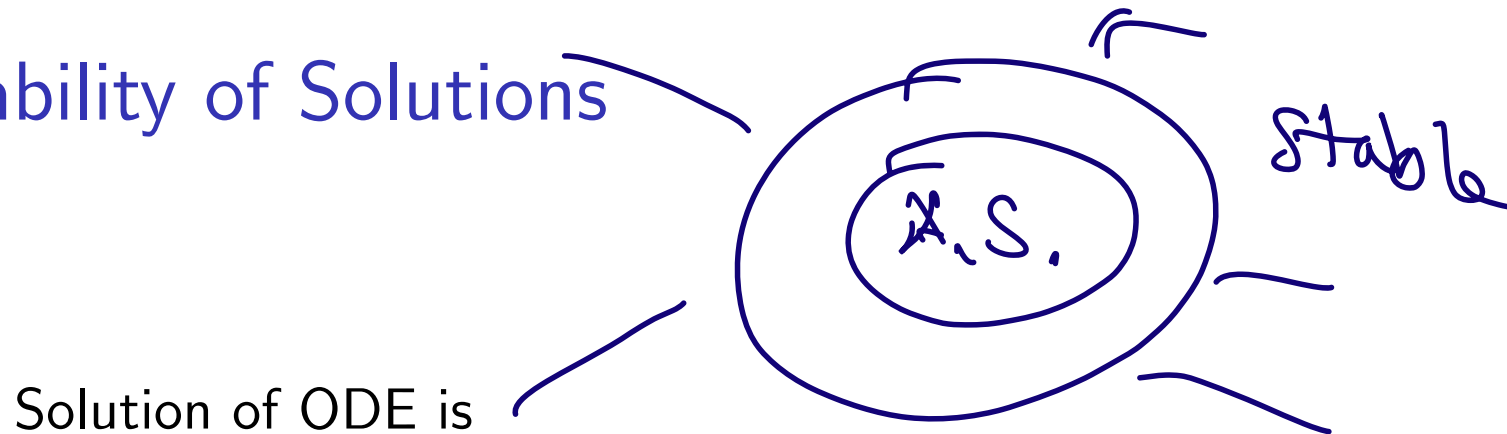
$$y(t_0) = y_0.$$

$$y' = y$$



Stability of ODE Solutions

Stability of Solutions

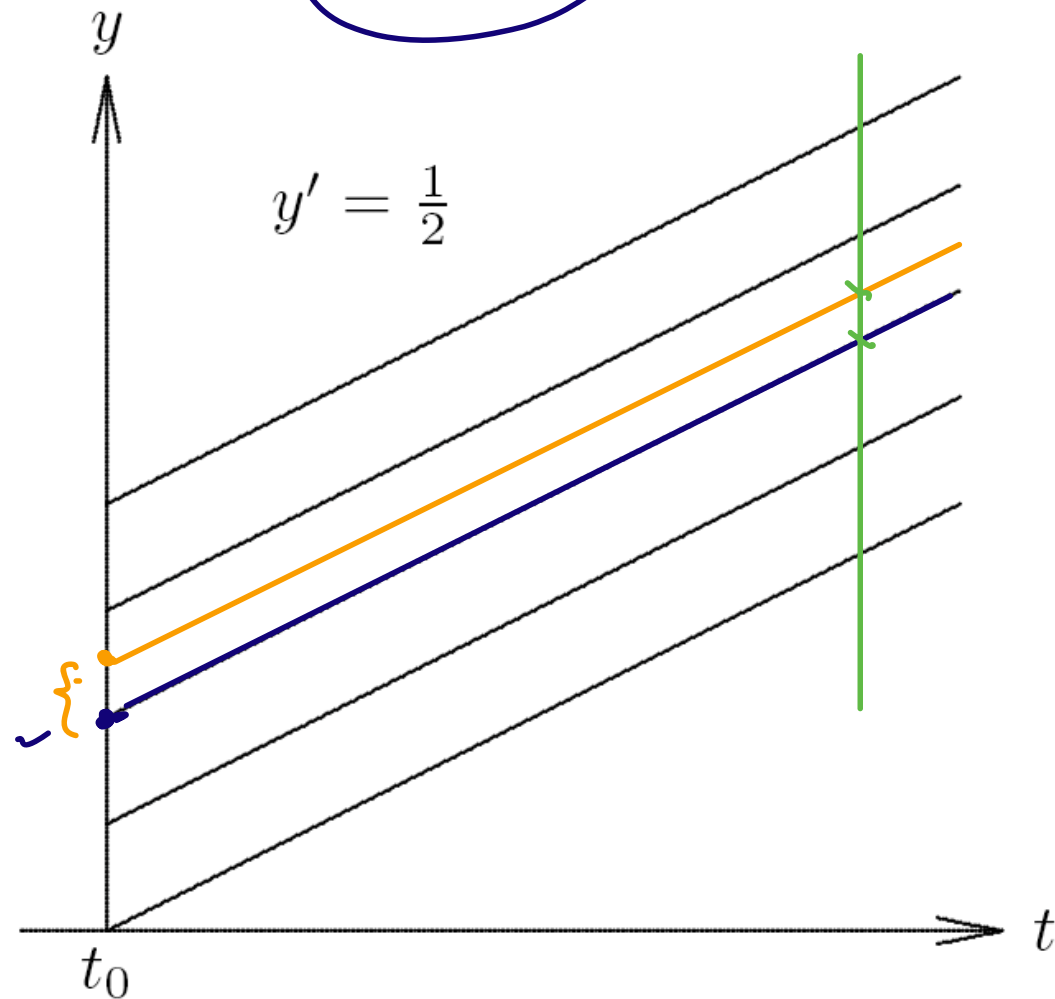


Solution of ODE is

- ▶ Stable if solutions resulting from perturbations of initial value remain close to original solution
- ▶ *Asymptotically stable* if solutions resulting from perturbations converge back to original solution
- ▶ *Unstable* if solutions resulting from perturbations diverge away from original solution without bound

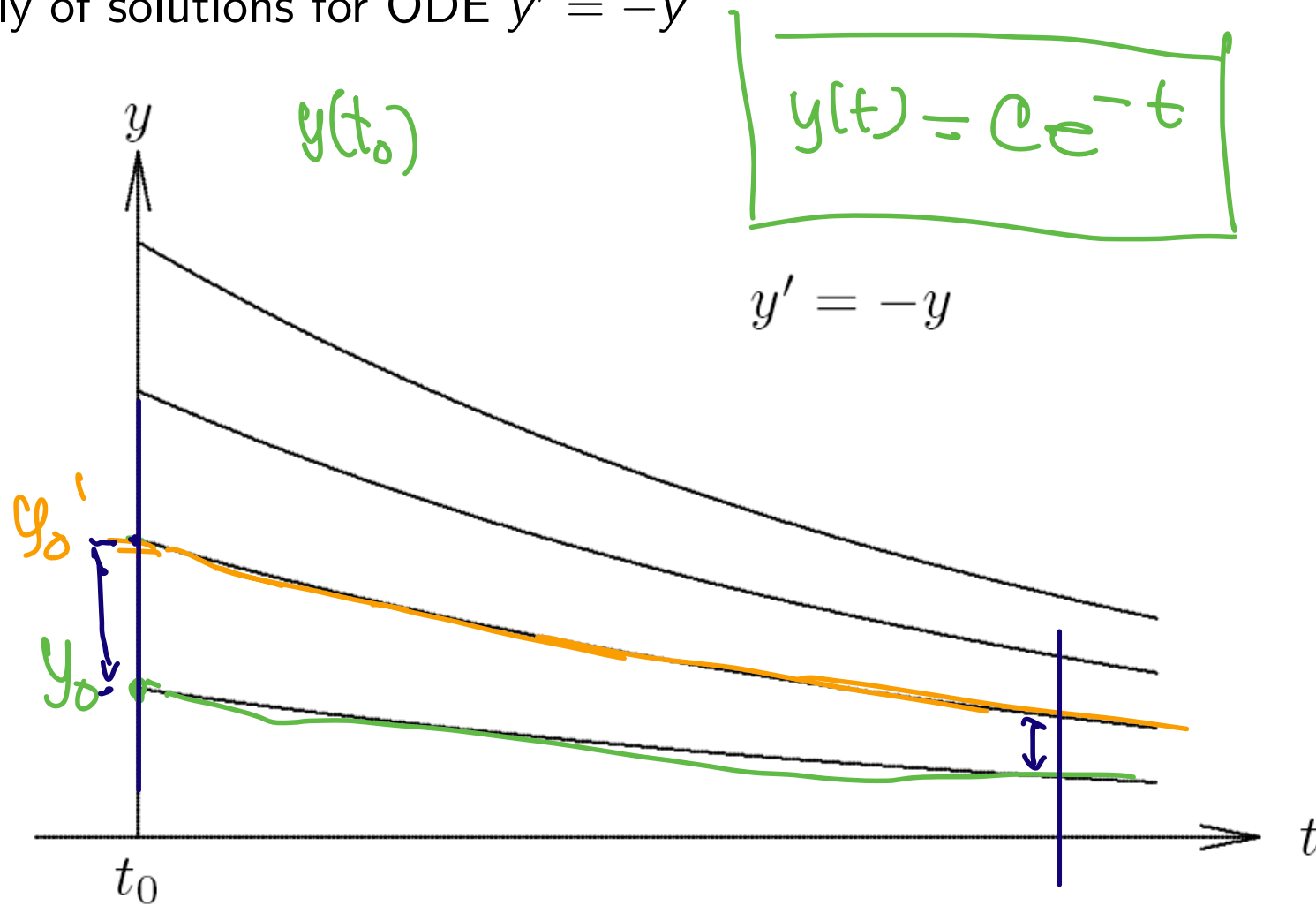
Example: Stable Solutions

Family of solutions for ODE $y' = \frac{1}{2}$



Example: Asymptotically Stable Solutions

Family of solutions for ODE $y' = -y$



Example: Scalar Test ODE

- ▶ Consider scalar ODE $y' = \lambda y$, where λ is constant.
- ▶ Solution is given by $y(t) = y_0 e^{\lambda t}$, where $t_0 = 0$ is initial time and $y(0) = y_0$ is initial value

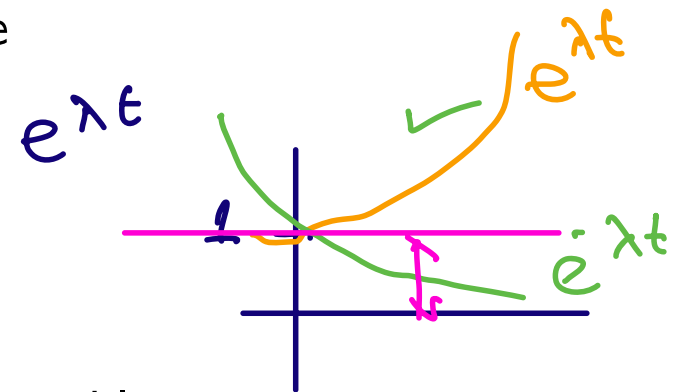
$$\int \frac{y'}{y} dt = \int \lambda dy$$

$$\ln(y) = \lambda t + C$$

$$y = y_0 e^{\lambda t}$$

- ▶ For real λ
 - ▶ $\lambda > 0$: all nonzero solutions grow exponentially, so every solution is unstable
 - ▶ $\lambda < 0$: all nonzero solutions decay exponentially, so every solution is not only stable, but asymptotically stable

- ▶ For complex λ
 - ▶ $\text{Re}(\lambda) > 0$: unstable
 - ▶ $\text{Re}(\lambda) < 0$: asymptotically stable
 - ▶ $\text{Re}(\lambda) = 0$: stable but not asymptotically stable



Example: Linear System of ODEs

$$y = \alpha_1 v_1 + \dots + \alpha_n v_n$$

$$y: \mathbb{R} \rightarrow \mathbb{R}^n$$

- Linear, homogeneous system of ODEs with constant coefficients has form

$$y' = Ay$$

$$y(t) \in \mathbb{R}^n$$

$$y'(t) \in \mathbb{R}^n$$

where A is $n \times n$ matrix, and initial condition is $y(0) = y_0$

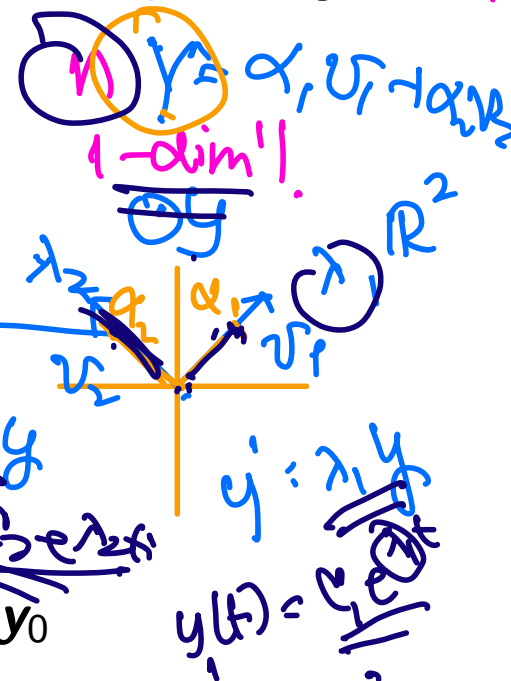
- Suppose A is diagonalizable, with eigenvalues λ_i and corresponding eigenvectors v_i , $i = 1, \dots, n$

- Express y_0 as linear combination $y_0 = \sum_{i=1}^n \alpha_i v_i$

- Then

$$y(t) = \sum_{i=1}^n \alpha_i v_i e^{\lambda_i t}$$

is solution to ODE satisfying initial condition $y(0) = y_0$



Example, continued

- ▶ Eigenvalues of $\mathbf{A}$ with positive real parts yield exponentially growing solution components
- ▶ Eigenvalues with negative real parts yield exponentially decaying solution components
- ▶ Eigenvalues with zero real parts (i.e., pure imaginary) yield oscillatory solution components
- ▶ Solutions stable if $\text{Re}(\lambda_i) \leq 0$ for every eigenvalue, and asymptotically stable if $\text{Re}(\lambda_i) < 0$ for every eigenvalue, but unstable if $\text{Re}(\lambda_i) > 0$ for any eigenvalue

$y' = \mathbf{A}y$ $\rightarrow$ linear fcn $y' = f(y)$ $\neq$ lin.

Stability of Solutions, General Case

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m, f \in C^k$$

$$\frac{df}{dy} \in \mathbb{R}^{m \times n}$$

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots + \frac{f^{(k)}(a)}{k!}(x-a)^k + O((x-a)^{k+1})$$

- For general nonlinear system of ODEs $y' = f(t, y)$, determining stability of solutions is more complicated

Linearize!

- ODE can be linearized locally about solution $y(t)$ by truncated Taylor series, yielding linear ODE

$$z' = J_f(t, y(t)) z$$

where J_f is Jacobian matrix of f with respect to y

$$\begin{bmatrix} J_f \end{bmatrix} \begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix}$$

- Eigenvalues of J_f determine stability locally, but conclusions drawn may not be valid globally

1st derivative — gradient — Jacobian
2nd derivative — Hessian

Euler's Method

Numerical Solution of ODEs

- ▶ Analytical solution of ODE is closed-form formula that can be evaluated at any point t
- ▶ Numerical solution of ODE is table of approximate values of solution function at discrete set of points
- ▶ Numerical solution is generated by simulating behavior of system governed by ODE
- ▶ Starting at t_0 with given initial value $\mathbf{y}_0$, we track trajectory dictated by ODE
- ▶ Evaluating $\mathbf{f}(t_0, \mathbf{y}_0)$ tells us slope of trajectory at that point
- ▶ We use this information to predict value $\mathbf{y}_1$ of solution at future time $t_1 = t_0 + h$ for some suitably chosen time increment h

Numerical Solution of ODEs, continued

- ▶ Approximate solution values are generated step by step in increments moving across interval in which solution is sought
- ▶ In stepping from one discrete point to next, we incur some error, which means that next approximate solution value lies on *different* solution from one we started on
- ▶ Stability or instability of solutions determines, in part, whether such errors are magnified or diminished with time

Euler's Method

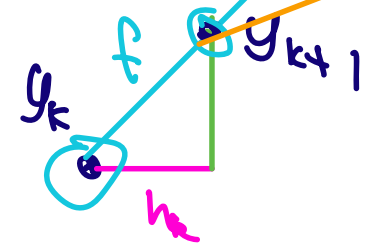


- For general system of ODEs $y' = f(t, y)$, consider Taylor series

$$\begin{aligned}
 y(t+h) &= y(t) + hy'(t) + \frac{h^2}{2}y''(t) + \dots \\
 &= y(t) + hf(t, y(t)) + \frac{h^2}{2}y''(t) + \dots
 \end{aligned}$$

- *Euler's method* results from dropping terms of second and higher order to obtain approximate solution value

$$y_{k+1} = y_k + h_k f(t_k, y_k)$$



- Euler's method advances solution by extrapolating along straight line whose slope is given by $f(t_k, y_k)$
- Euler's method is *single-step* method because it depends on information at only one point in time to advance to next point

Example: Euler's Method

$$y(t_0) = y_0$$

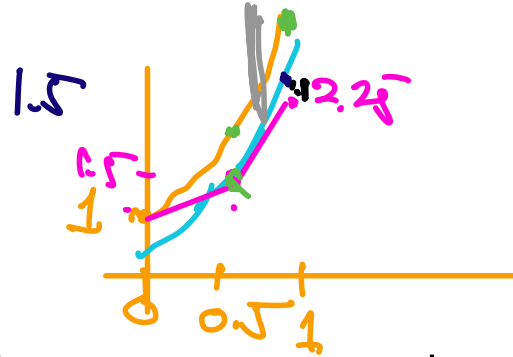
- ▶ Applying Euler's method to ODE $y' = y$ with step size h , we advance solution from time $t_0 = 0$ to time $t_1 = t_0 + h$

$$\checkmark \quad y_1 = y_0 + h y_0' = y_0 + h y_0 = (1 + h) y_0$$

- ▶ Value for solution we obtain at t_1 is not exact, $y_1 \neq y(t_1)$
- ▶ For example, if $t_0 = 0$, $y_0 = 1$, and $h = 0.5$, then $y_1 = 1.5$, whereas exact solution for this initial value is $y(0.5) = \exp(0.5) \approx 1.649$
- ▶ Thus, y_1 lies on different solution from one we started on

Example, continued

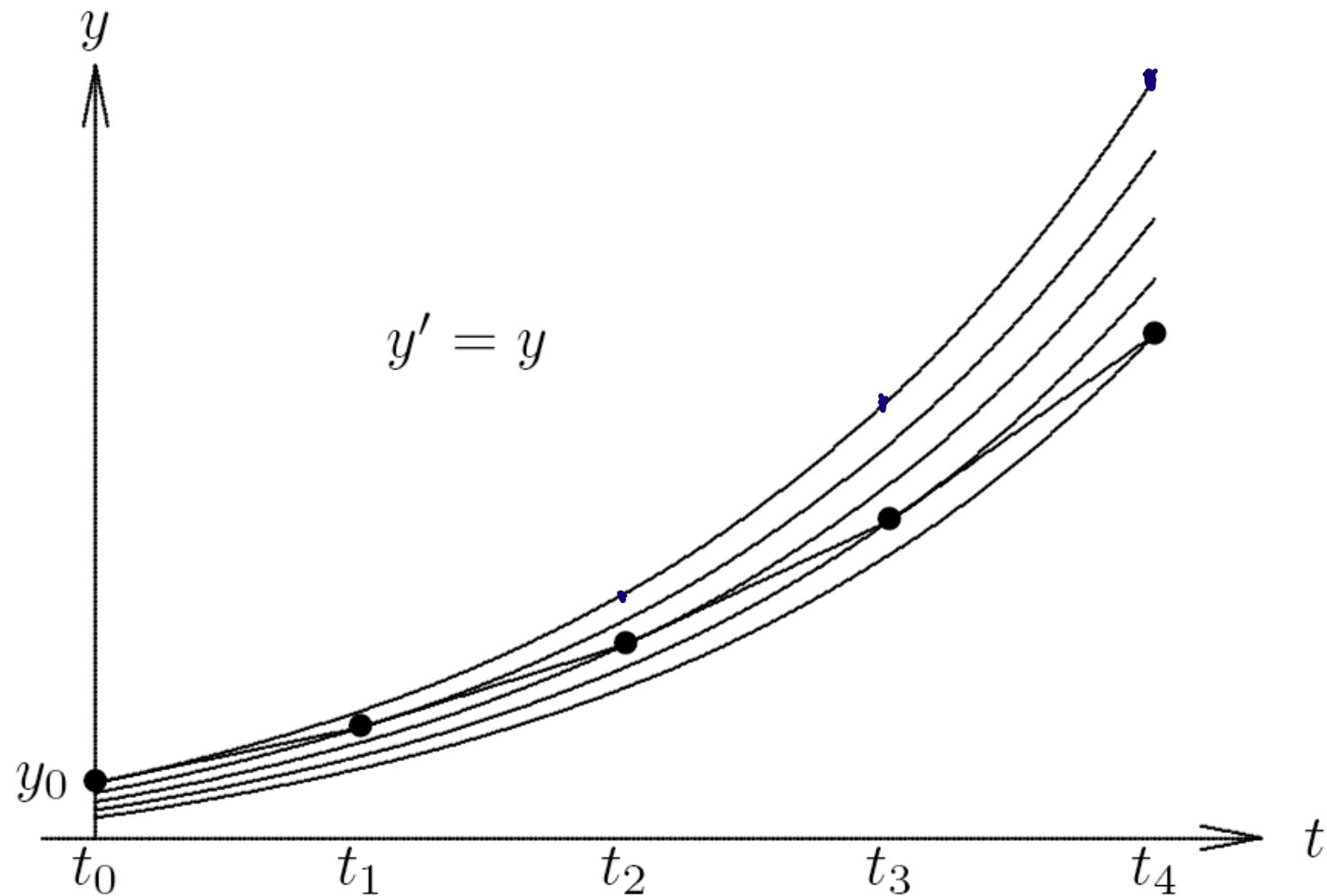
$$\begin{cases} y' = y \\ y(0) = 1 \end{cases}$$



- ▶ To continue numerical solution process, we take another step from t_1 to $t_2 = t_1 + h = 1.0$, obtaining

$$y_2 = y_1 + hy_1 = 1.5 + (0.5)(1.5) = \underline{2.25}$$
- ▶ Now y_2 differs not only from true solution of original problem at $t = 1$, $y(1) = \exp(1) \approx 2.718$, but it also differs from solution through previous point $(\underline{t_1}, y_1)$, which has approximate value 2.473 at $t = 1$
- ▶ Thus, we have moved to still another solution for this ODE

Example: Unstable Solutions



For unstable solutions, errors in numerical solution grow with time

Errors in Numerical Solution of ODEs

- ▶ Numerical methods for solving ODEs incur two distinct types of error
 - ▶ *Rounding error*, which is due to finite precision of floating-point arithmetic
 - ▶ *Truncation error* (*discretization error*), which is due to approximation method used and would remain even if all arithmetic were exact
- ▶ In practice, truncation error is dominant factor determining accuracy of numerical solutions of ODEs, so we will henceforth ignore rounding error

Global Error and Local Error

Truncation error at any point t_k can be broken down into

- ▶ *Global error*: difference between computed solution and true solution $\mathbf{y}(t)$ passing through initial point $(t_0, \mathbf{y}_0)$

$$\mathbf{e}_k = \mathbf{y}_k - \mathbf{y}(t_k)$$

- ▶ *Local error*: error made in one step of numerical method

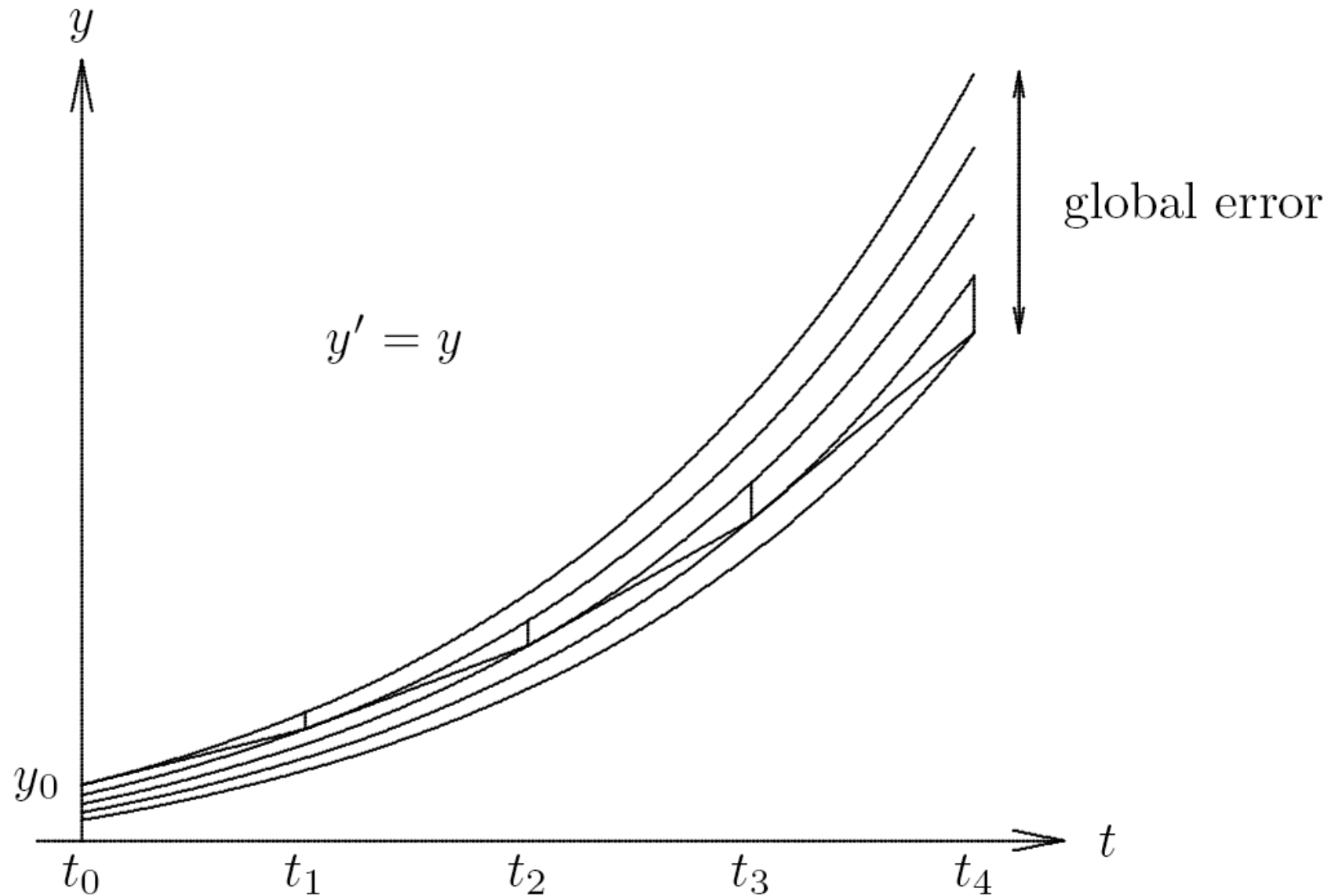
$$\ell_k = \mathbf{y}_k - \mathbf{u}_{k-1}(t_k)$$

where $\mathbf{u}_{k-1}(t)$ is true solution passing through previous point $(t_{k-1}, \mathbf{y}_{k-1})$

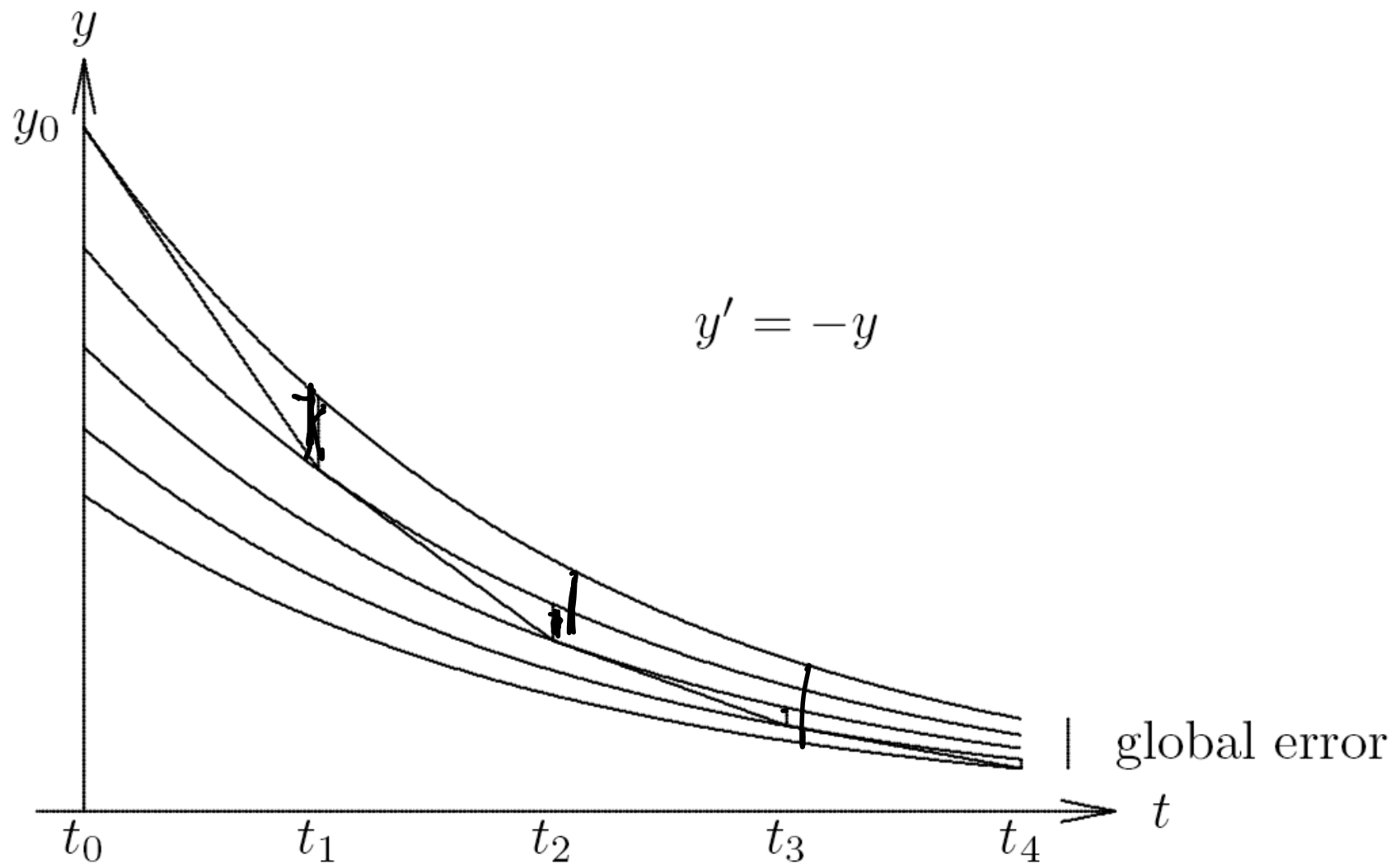
Global Error and Local Error, continued

- ▶ Global error is not necessarily sum of local errors
- ▶ Global error is generally greater than sum of local errors if solutions are unstable, but may be less than sum if solutions are stable
- ▶ We want small global error, but we can control only local error directly

Example: Unstable Solutions

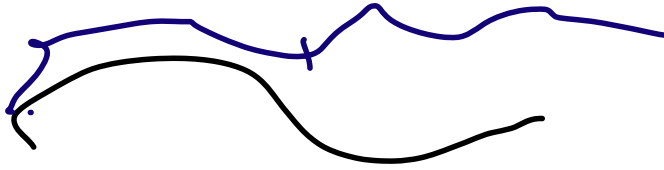


Example: Stable Solutions



Accuracy and Stability

Order of Accuracy



- ▶ *Order of accuracy* of numerical method is p if

$$\ell_k = \mathcal{O}(h_k^{p+1})$$

- ▶ Then local error per unit step, $\ell_k/h_k = \mathcal{O}(h_k^p)$
- ▶ Under reasonable conditions, $\mathbf{e}_k = \mathcal{O}(h^p)$, where h is average step size

⟨ interactive example ⟩

Stability of Numerical Method

- ▶ Numerical method is *stable* if small perturbations do *not* cause resulting numerical solutions to diverge from each other without bound
- ▶ Such divergence of numerical solutions could be caused by instability of solutions to ODE, but can also be due to numerical method itself, even when solutions to ODE are stable

Determining Stability and Accuracy $\lambda = a + bi$

$$y(t) = A e^{\alpha t} (\cos(\beta t) + \sin(\beta t))$$

- ▶ Simple approach to determining stability and accuracy of numerical method is to apply it to scalar ODE $y' = \lambda y$, where λ is (possibly complex) constant
- ▶ Exact solution is given by $y(t) = y_0 e^{\lambda t}$, where $y(0) = y_0$ is initial condition
- ▶ Determine stability of numerical method by characterizing growth of numerical solution
- ▶ Determine accuracy of numerical method by comparing exact and numerical solutions

Example: Euler's Method

- ▶ Applying Euler's method to $y' = \lambda y$ using fixed step size h ,

$$y_{k+1} = y_k + h\lambda y_k = (1 + h\lambda)y_k$$

which means that

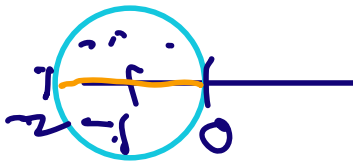
$$y_k = (1 + h\lambda)^k y_0$$

- ▶ If $\text{Re}(\lambda) < 0$, exact solution decays to zero as t increases, as does computed solution if

$$|1 + h\lambda| < 1$$

which holds if $h\lambda$ lies inside circle in complex plane of radius 1 centered at -1

$h\lambda$



< interactive example >

Euler's Method, continued

- ▶ If λ is real, then $h\lambda$ must lie in interval $(-2, 0)$, so for $\lambda < 0$, we must have

$$h \leq -\frac{2}{\lambda}$$

for Euler's method to be stable

- ▶ *Growth factor* $1 + h\lambda$ agrees with series expansion

$$e^{h\lambda} = 1 + h\lambda + \frac{(h\lambda)^2}{2} + \frac{(h\lambda)^3}{6} + \dots$$

through terms of first order in h , so Euler's method is first-order accurate

Euler's Method, continued

- ▶ For general system of ODEs $\mathbf{y}' = \mathbf{f}(t, \mathbf{y})$, consider Taylor series

$$\begin{aligned}\mathbf{y}(t+h) &= \mathbf{y}(t) + h\mathbf{y}'(t) + \mathcal{O}(h^2) \\ &= \mathbf{y}(t) + h\mathbf{f}(t, \mathbf{y}(t)) + \mathcal{O}(h^2)\end{aligned}$$

- ▶ If we take $t = t_k$ and $h = h_k$, we obtain

$$\mathbf{y}(t_{k+1}) = \mathbf{y}(t_k) + h_k \mathbf{f}(t_k, \mathbf{y}(t_k)) + \mathcal{O}(h_k^2)$$

- ▶ Subtracting this from Euler's method,

$$\begin{aligned}e_{k+1} &= \mathbf{y}_{k+1} - \mathbf{y}(t_{k+1}) \\ &= [\mathbf{y}_k - \mathbf{y}(t_k)] + h_k [\mathbf{f}(t_k, \mathbf{y}_k) - \mathbf{f}(t_k, \mathbf{y}(t_k))] + \mathcal{O}(h_k^2)\end{aligned}$$

- ▶ If there were no prior errors, then $\mathbf{y}_k = \mathbf{y}(t_k)$, and differences in brackets on right side would be zero, leaving only $\mathcal{O}(h_k^2)$ term, which is local error
- ▶ So Euler's method is first-order accurate

Euler's Method, continued

- ▶ From previous derivation, global error is sum of local error and *propagated* error

- ▶ From Mean Value Theorem,

$$f(t_k, y_k) - f(t_k, y(t_k)) = J_f(t_k, \xi)(y_k - y(t_k))$$

for some (unknown) value ξ , where J_f is Jacobian matrix of f with respect to y

- ▶ So we can express global error at step $k + 1$ as

$$e_{k+1} = (I + h_k J_f) e_k + \ell_{k+1}$$

- ▶ Thus, global error is multiplied at each step by *growth factor* $I + h_k J_f$

- ▶ Errors do not grow if spectral radius $\rho(I + h_k J_f) \leq 1$, which holds if all eigenvalues of $h_k J_f$ lie inside circle in complex plane of radius 1 centered at -1

Stability of Numerical Methods for ODEs

$$y' = f(y)$$

In general, growth factor depends on

- ▶ Numerical method, which determines form of growth factor
- ▶ Jacobian $\mathbf{J}_f$, which is determined by particular ODE
- ▶ Step size h

$$\begin{pmatrix} y_{n+1} \\ \vdots \end{pmatrix} = \mathbf{G} \begin{pmatrix} y_n \\ \vdots \end{pmatrix}$$

Step Size Selection

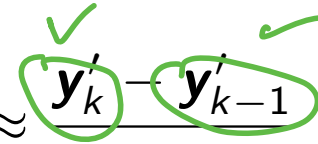
- ▶ In choosing step size for advancing numerical solution of ODE, we want to take large steps to reduce computational cost, but must also take into account both stability and accuracy
- ▶ To yield meaningful solution, step size must obey any stability restrictions
- ▶ In addition, local error estimate is needed to ensure that desired accuracy is achieved
- ▶ With Euler's method, for example, local error is approximately $(h_k^2/2)\mathbf{y}''$, so choose step size to satisfy

error

$$h_k \leq \sqrt{2 \text{tol} / \|\mathbf{y}''\|}$$

Step Size Selection, continued

- ▶ We do not know value of $\mathbf{y}''$, but we can estimate it by difference quotient

$$\mathbf{y}'' \approx \frac{\mathbf{y}'_k - \mathbf{y}'_{k-1}}{t_k - t_{k-1}}$$


- ▶ Other methods of obtaining error estimates are based on difference between results obtained using methods of different orders or different step sizes

Euler

Brw an 2

$$\frac{y_{k+1} - y_k}{h} = f(y_k)$$

$\begin{array}{c} \textcircled{y'} = f(y) \\ \downarrow \end{array}$

$$\Rightarrow y_{k+1} = y_k + h f(y_k)$$

Implicit Methods

$$\frac{y_k - y_{k-1}}{h} = f(y_k)$$

$$y_k = y_{k-1} + h f(y_k)$$

Implicit Methods

- ▶ Euler's method is *explicit* in that it uses only information at time t_k to advance solution to time t_{k+1}
- ▶ This may seem desirable, but Euler's method has rather limited stability region
- ▶ Larger stability region can be obtained by using information at time t_{k+1} , which makes method *implicit*

- ▶ Simplest example is *backward Euler method*

$$y^* = y_k + h_k f(t, y^*)$$

$$\underline{y_{k+1}} = \underline{y_k} + h_k \underline{f(t_{k+1}, y_{k+1})}$$

Handwritten notes: y_{k+1}^* above the second equation, and green circles and arrows highlighting the implicit nature of the backward Euler method.

- ▶ Method is implicit because we must evaluate f with argument y_{k+1} before we know its value

Implicit Methods, continued

- ▶ This means that we must solve algebraic equation to determine $\mathbf{y}_{k+1}$
- ▶ Typically, we use iterative method such as Newton's method or fixed-point iteration to solve for $\mathbf{y}_{k+1}$
- ▶ Good starting guess for iteration can be obtained from explicit method, such as Euler's method, or from solution at previous time step

⟨ interactive example ⟩

Example: Backward Euler Method

- ▶ Consider nonlinear scalar ODE $y' = -y^3$ with initial condition $y(0) = 1$

- ▶ Using backward Euler method with step size $h = 0.5$, we obtain implicit equation

$$y_1 = y_0 + hf(t_1, y_1) = 1 - 0.5y_1^3$$

for solution value at next step

- ▶ This nonlinear equation for y_1 could be solved by fixed-point iteration or Newton's method
- ▶ To obtain starting guess for y_1 , we could use previous solution value, $y_0 = 1$, or we could use explicit method, such as Euler's method, which gives $y_1 = y_0 - 0.5y_0^3 = 0.5$
- ▶ Iterations eventually converge to final value $y_1 \approx 0.7709$

Implicit Methods, continued

- ▶ Given extra trouble and computation in using implicit method, one might wonder why we bother
- ▶ Answer is that implicit methods generally have significantly larger stability region than comparable explicit methods

Backward Euler Method

- ▶ To determine stability of backward Euler, we apply it to scalar ODE $y' = \lambda y$, obtaining

$$y_{k+1} = y_k + h\lambda y_{k+1}$$

$$(1 - h\lambda)y_{k+1} = y_k$$

$$y_k = \left(\frac{1}{1 - h\lambda} \right)^k y_0$$

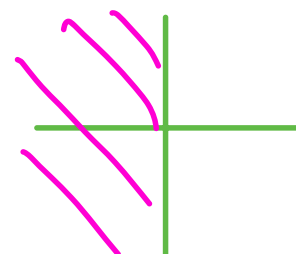
$$y_k = \lambda y_{k-1}$$

$$= \lambda^k y_0$$

- ▶ Thus, for backward Euler to be stable we must have

$$\left| \frac{1}{1 - h\lambda} \right| \leq 1$$

✓



λ

which holds for any $h > 0$ when $\text{Re}(\lambda) < 0$

- ▶ So stability region for backward Euler method includes entire left half of complex plane, or interval $(-\infty, 0)$ if λ is real

$(0, +\infty)$

Backward Euler Method, continued

- Growth factor

$$\frac{1}{1 - h\lambda} = 1 + h\lambda + (h\lambda)^2 + \dots$$

agrees with expansion for $e^{\lambda h}$ through terms of order h , so backward Euler method is first-order accurate

- Growth factor of backward Euler method for general system of ODEs $\mathbf{y}' = \mathbf{f}(t, \mathbf{y})$ is $(\mathbf{I} - h\mathbf{J}_f)^{-1}$, whose spectral radius is less than 1 provided all eigenvalues of $h\mathbf{J}_f$ lie outside circle in complex plane of radius 1 centered at 1
- Thus, stability region of backward Euler for general system of ODEs is entire left half of complex plane

Unconditionally Stable Methods

- ▶ Thus, for computing stable solution backward Euler is stable for any positive step size, which means that it is *unconditionally* stable
- ▶ Great virtue of unconditionally stable method is that desired accuracy is only constraint on choice of step size
- ▶ Thus, we may be able to take much larger steps than for explicit method of comparable order and attain much higher overall efficiency despite requiring more computation per step
- ▶ Although backward Euler method is unconditionally stable, its accuracy is only of first order, which severely limits its usefulness

Trapezoid Method

- Higher-order accuracy can be achieved by averaging Euler and backward Euler methods to obtain implicit *trapezoid method*

$$\mathbf{y}_{k+1} = \mathbf{y}_k + h_k (\mathbf{f}(t_k, \mathbf{y}_k) + \mathbf{f}(t_{k+1}, \mathbf{y}_{k+1})) / 2$$

- To determine its stability and accuracy, we apply it to scalar ODE $y' = \lambda y$, obtaining

$$y_{k+1} = y_k + h (\lambda y_k + \lambda y_{k+1}) / 2$$

$$y_k = \left(\frac{1 + h\lambda/2}{1 - h\lambda/2} \right)^k y_0$$

- Method is stable if

$$\left| \frac{1 + h\lambda/2}{1 - h\lambda/2} \right| < 1$$

which holds for any $h > 0$ when $\text{Re}(\lambda) < 0$

• E: $y_{k+1} = y_k + h f(y_k)$
 • B: $y_{k+1} = y_k + h f(y_{k+1})$
 • T: $y_{k+1} = y_k + h \frac{f(y_k) + f(y_{k+1})}{2}$

L:

Trapezoid Method, continued

- ▶ Thus, trapezoid method is unconditionally stable
- ▶ Its growth factor

$$\begin{aligned}\frac{1 + h\lambda/2}{1 - h\lambda/2} &= \left(1 + \frac{h\lambda}{2}\right) \left(1 + \frac{h\lambda}{2} + \left(\frac{h\lambda}{2}\right)^2 + \left(\frac{h\lambda}{2}\right)^3 + \dots\right) \\ &= 1 + h\lambda + \frac{(h\lambda)^2}{2} + \frac{(h\lambda)^3}{4} + \dots\end{aligned}$$

agrees with expansion of $e^{h\lambda}$ through terms of order h^2 , so trapezoid method is second-order accurate

- ▶ For general system of ODEs $\mathbf{y}' = \mathbf{f}(t, \mathbf{y})$, trapezoid method has growth factor $(\mathbf{I} + \frac{1}{2}h\mathbf{J}_f)(\mathbf{I} - \frac{1}{2}h\mathbf{J}_f)^{-1}$, whose spectral radius is less than 1 provided eigenvalues of $h\mathbf{J}_f$ lie in left half of complex plane

⟨ interactive example ⟩



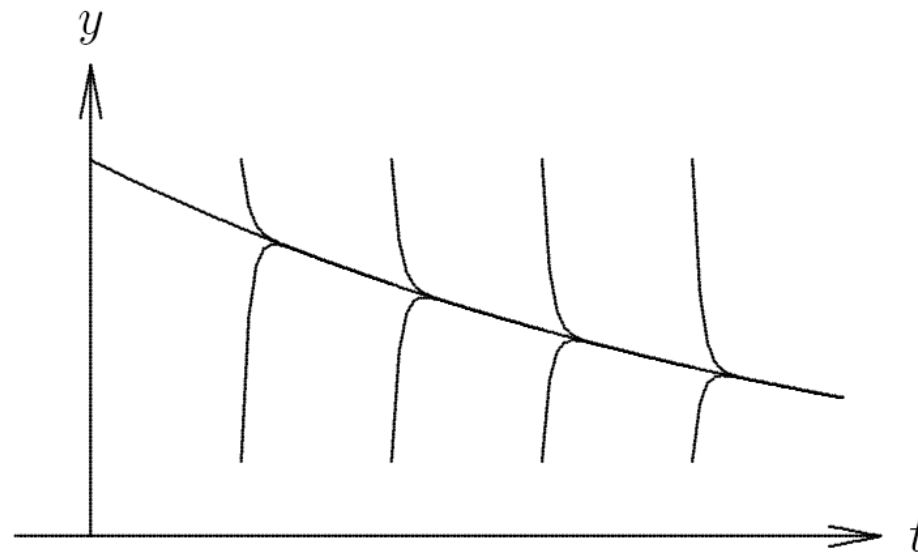
Implicit Methods, continued

- ▶ We have now seen two examples of implicit methods that are unconditionally stable, but not all implicit methods have this property
- ▶ Implicit methods generally have larger stability regions than explicit methods, but allowable step size is not always unlimited
- ▶ Implicitness alone is not sufficient to guarantee stability

Stiffness

Stiff ODEs

- ▶ Asymptotically stable solutions converge with time, and this has favorable property of damping errors in numerical solution
- ▶ But if convergence of solutions is too rapid, then difficulties of different type may arise
- ▶ Such ODE is said to be *stiff*



$$\lambda = a + bi$$

Stiff ODEs, continued

$$A e^{\lambda t}$$

$$x(t) = A e^{at} [\cos(bt) + i \sin(bt)]$$

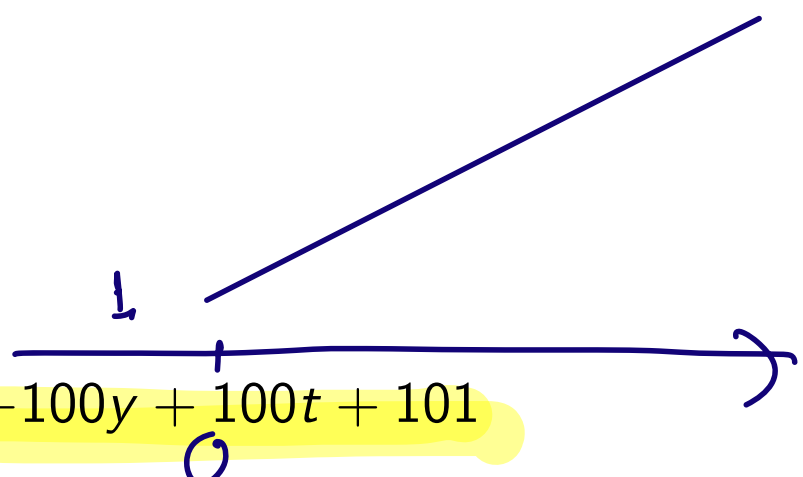
- ▶ Stiff ODE corresponds to physical process whose components have disparate time scales or whose time scale is small compared to interval over which it is studied
- ▶ System of ODEs $y' = f(t, y)$ is stiff if eigenvalues of its Jacobian matrix J_f differ greatly in magnitude
- ▶ There may be eigenvalues with
 - ✓ large negative real parts, corresponding to strongly damped components of solution, or
 - ▶ large imaginary parts, corresponding to rapidly oscillating components of solution

Stiff ODEs, continued

- ▶ Some numerical methods are inefficient for stiff ODEs because rapidly varying component of solution forces very small step sizes to maintain stability
- ▶ Stability restriction depends on rapidly varying component of solution, but accuracy restriction depends on slowly varying component, so step size may be more severely restricted by stability than by required accuracy
- ▶ For example, Euler's method is extremely inefficient for solving stiff ODEs because of severe stability limitation on step size
- ▶ **Backward Euler** method is suitable for stiff ODEs because of its unconditional stability
- ▶ Stiff ODEs need not be difficult to solve numerically, provided suitable method is chosen

Example: Stiff ODE

- Consider scalar ODE

$$y' = -100y + 100t + 101$$


with initial condition $y(0) = 1$

- General solution is $y(t) = 1 + t + ce^{-100t}$, and particular solution satisfying initial condition is $y(t) = 1 + t$ (i.e., $c = 0$)
- Since solution is linear, Euler's method is theoretically exact for this problem
- However, let us perturb initial value slightly

Example, continued

$$h \leq \frac{2}{|\lambda|} \times = 0.02$$

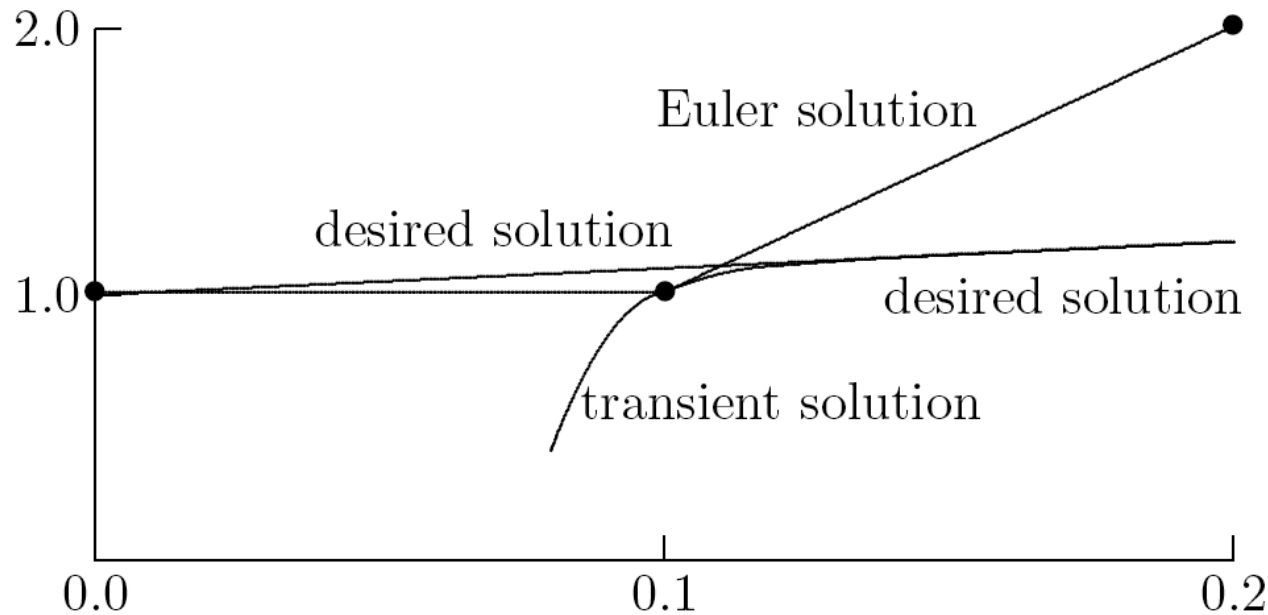
- ▶ With step size $h = 0.1$, first few steps for given initial values are

t	0.0	0.1	0.2	0.3	0.4
exact sol.	1.00	1.10	1.20	1.30	1.40
Euler sol.	0.99	1.19	0.39	8.59	-64.2
Euler sol.	1.01	1.01	2.01	-5.99	67.0

- ▶ Computed solution is incredibly sensitive to initial value, as each tiny perturbation results in wildly different solution
- ▶ Any point deviating from desired particular solution, even by only small amount, lies on different solution, for which $c \neq 0$, and therefore rapid transient of general solution is present

⟨ interactive example ⟩

Example, continued



- ▶ Euler's method bases its projection on derivative at current point, and resulting large value causes numerical solution to diverge radically from desired solution
- ▶ Jacobian for this ODE is $J_f = -100$, so stability condition for Euler's method requires step size $h < 0.02$, which we are violating

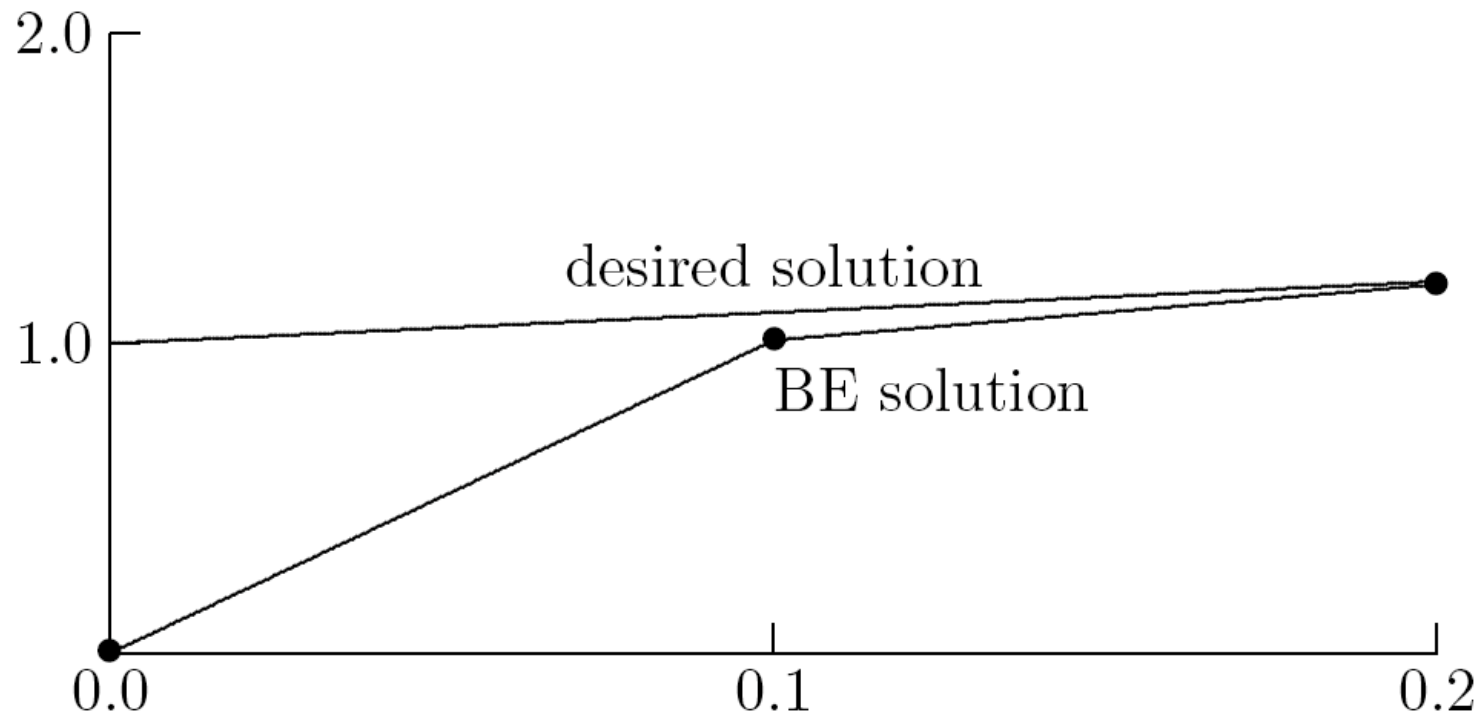
Example, continued

- ▶ Backward Euler method has no trouble solving this problem and is extremely *insensitive* to initial value

t	0.0	0.1	0.2	0.3	0.4
exact sol.	1.00	1.10	1.20	1.30	1.40
BE sol.	0.00	1.01	1.19	1.30	1.40
BE sol.	2.00	1.19	1.21	1.30	1.40

- ▶ Even with very large perturbation in initial value, by using derivative at next point rather than current point, transient is quickly damped out and backward Euler solution converges to desired solution after only few steps
- ▶ This behavior is consistent with unconditional stability of backward Euler method for stable solutions

Example, continued



⟨ interactive example ⟩

Single-Step Methods

Numerical Methods for ODEs

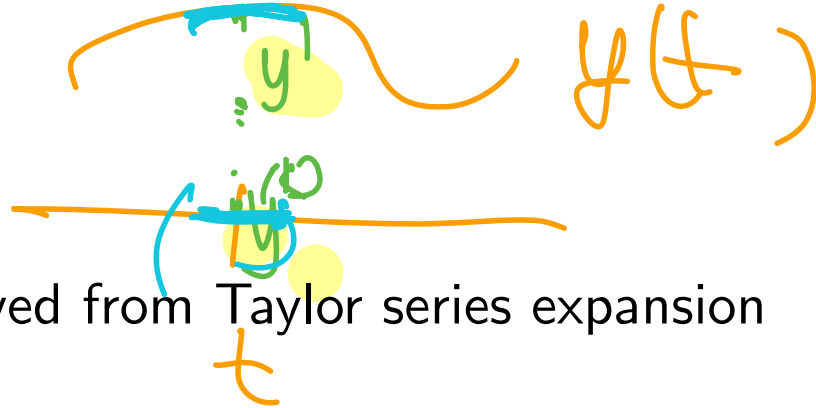
There are many different methods for solving ODEs, most of which are of these types

- ▶ Single-step
 - ▶ Taylor series
 - ▶ Runge-Kutta
 - ▶ Extrapolation
- ▶ Multistep
- ▶ Multivalued

We briefly consider each of these types of methods

Taylor Series Methods

polynomial



- ▶ Euler's method can be derived from Taylor series expansion
- ▶ By retaining more terms in Taylor series, we can generate single-step methods of higher-order
- ▶ For example, retaining one additional term in Taylor series

$t = t_k$
 $t_k + h = t_{k+1}$

$$y(t+h) = y(t) + h y'(t) + \frac{h^2}{2} y''(t) + \frac{h^3}{6} y'''(t) + \dots$$

gives second-order method

$$y_{k+1} = y_k + h_k y'_k + \frac{h_k^2}{2} y''_k$$

✓

Taylor Series Methods, continued

- This approach requires computation of higher derivatives of $\mathbf{y}$, which can be obtained by differentiating $\mathbf{y}' = \mathbf{f}(t, \mathbf{y})$ using chain rule, e.g.,

$$\begin{aligned} \mathbf{y}'' &= \mathbf{f}_t(t, \mathbf{y}) + \mathbf{f}_y(t, \mathbf{y}) \mathbf{y}' \\ &= \mathbf{f}_t(t, \mathbf{y}) + \mathbf{f}_y(t, \mathbf{y}) \mathbf{f}(t, \mathbf{y}) \end{aligned}$$

where subscripts indicate partial derivatives with respect to given variable

- As order increases, expressions for derivatives rapidly become too complicated to be practical to compute, so Taylor series methods of higher order have not often been used in practice

⟨ interactive example ⟩

Runge-Kutta Methods

- ▶ *Runge-Kutta methods* are single-step methods similar in motivation to Taylor series methods, but they do not require computation of higher derivatives
- ▶ Instead, Runge-Kutta methods simulate effect of higher derivatives by evaluating $\mathbf{f}$ several times between t_k and t_{k+1}

Heun's Method

$$y_{k+1} = y_k + h \underbrace{(\text{slope})}_{\text{coverage of slope!}}$$

$f(y_k)$

- Simplest example is second-order Heun's method

$$y_{k+1} = y_k + \frac{h_k}{2} (k_1 + k_2)$$

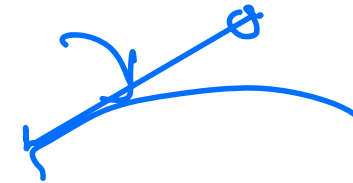
where

$$k_1 = f(t_k, y_k)$$

$$k_2 = f(t_k + h_k, \underline{y_k + h_k k_1})$$

$$k_2 = f(t_k + h_k, \underline{y_k + h_k k_1})$$

- Heun's method is analogous to implicit trapezoid method, but remains explicit by using Euler prediction $y_k + h_k k_1$ instead of $y(t_{k+1})$ in evaluating f at t_{k+1}



Classical Fourth-Order Runge-Kutta Method

- ▶ Best-known Runge-Kutta method is classical fourth-order scheme

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \frac{h_k}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

where

$$\mathbf{k}_1 = \mathbf{f}(t_k, \mathbf{y}_k)$$

$$\mathbf{k}_2 = \mathbf{f}(t_k + h_k/2, \mathbf{y}_k + (h_k/2)\mathbf{k}_1)$$

$$\mathbf{k}_3 = \mathbf{f}(t_k + h_k/2, \mathbf{y}_k + (h_k/2)\mathbf{k}_2)$$

$$\mathbf{k}_4 = \mathbf{f}(t_k + h_k, \mathbf{y}_k + h_k\mathbf{k}_3)$$

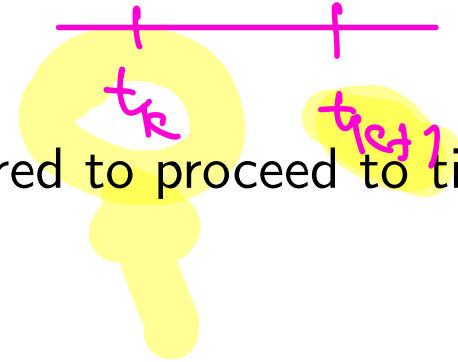
- ▶ Analogous to Simpson's rule

⟨ interactive example ⟩

Runge-Kutta Methods – Pros and Cons

Advantages of classical Runge-Kutta methods

- ▶ No history of solution prior to time t_k required to proceed to time t_{k+1}
 - ▶ *Self-starting* at beginning of integration
 - ▶ Easy to change step size during integration
- ▶ Relatively easy to program, which accounts in part for their popularity



Disadvantages of classical Runge-Kutta methods

- ▶ Provide no error estimate on which to base choice of step size
- ▶ Inefficient for stiff ODEs

Embedded Runge-Kutta Methods

- ▶ Fehlberg devised *embedded Runge-Kutta* method that uses six function evaluations per step to produce both fifth-order and fourth-order estimates of solution, whose difference provides estimate for local error
- ▶ Another embedded Runge-Kutta method is due to Dormand and Prince
- ▶ This approach has led to automatic Runge-Kutta solvers that are effective for many problems, but which are still relatively inefficient for stiff problems or when very high accuracy is required
- ▶ It is possible, however, to derive *implicit* Runge-Kutta methods with superior stability properties that are suitable for solving stiff ODEs

Extrapolation Methods

$$y_{k+1} = y_k + R$$

averaged slope

$$y_{k+1} = \frac{2f(0)}{h} - y_k$$

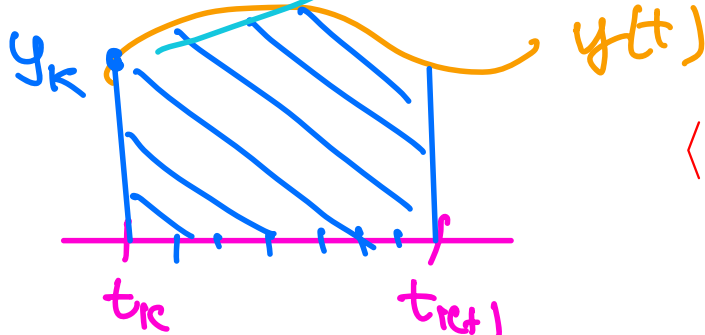
- ▶ **Extrapolation methods** are based on use of single-step method to integrate ODE over given interval $t_k \leq t \leq t_{k+1}$ using several different step sizes h_i , and yielding results denoted by $Y(h_i)$

- ▶ This gives discrete approximation to function $Y(h)$, where

$$Y(0) = y(t_{k+1})$$

- ▶ Interpolating polynomial or rational function $\hat{Y}(h)$ is fit to these data, and $\hat{Y}(0)$ is then taken as approximation to $Y(0)$

- ▶ Extrapolation methods are capable of achieving very high accuracy, but they are much less efficient and less flexible than other methods for ODEs, so they are not often used unless extremely high accuracy is required and cost is not significant factor



< interactive example >

$$R_n = \frac{t_{k+1} - t_k}{n}$$

$$R_n \rightarrow 0 \text{ as } n \rightarrow \infty$$

Multistep Methods

- ▶ *Multistep methods* use information at more than one previous point to estimate solution at next point
- ▶ *Linear multistep* methods have form

$$\mathbf{y}_{k+1} = \sum_{i=1}^m \alpha_i \mathbf{y}_{k+1-i} + h \sum_{i=0}^m \beta_i \mathbf{f}(t_{k+1-i}, \mathbf{y}_{k+1-i})$$

- ▶ Parameters α_i and β_i are determined by polynomial interpolation
- ▶ If $\beta_0 = 0$, method is explicit, but if $\beta_0 \neq 0$, method is implicit
- ▶ Implicit methods are usually more accurate and stable than explicit methods, but require starting guess for $\mathbf{y}_{k+1}$

Predictor-Corrector Methods

- ▶ Starting guess is conveniently supplied by explicit method, so the two are used as *predictor-corrector* pair
- ▶ One could use corrector repeatedly (i.e., fixed-point iteration) until some convergence tolerance is met, but it may not be worth expense
- ▶ So fixed number of corrector steps, often only one, may be used instead, giving *PECE* scheme (predict, evaluate, correct, evaluate)
- ▶ Although it has no effect on value of $\mathbf{y}_{k+1}$, second evaluation of $\mathbf{f}$ in PECE scheme yields improved value of $\mathbf{y}'_{k+1}$ for future use

Multistep Methods, continued

- ▶ Alternatively, nonlinear equation for $\mathbf{y}_{k+1}$ given by implicit multistep method can be solved by Newton's method or similar method, again with starting guess supplied by solution at previous step or by explicit multistep method
- ▶ Newton's method or close variant of it is essential when using an implicit multistep method designed for stiff ODEs, as fixed-point iteration fails to converge for reasonable step sizes

⟨ interactive example ⟩

Examples: Multistep Methods

- ▶ Simplest second-order accurate explicit two-step method is

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \frac{h}{2}(3\mathbf{y}'_k - \mathbf{y}'_{k-1})$$

- ▶ Simplest second-order accurate implicit method is *trapezoid method*

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \frac{h}{2}(\mathbf{y}'_{k+1} + \mathbf{y}'_k)$$

- ▶ One of most popular pairs of multistep methods is explicit fourth-order *Adams-Bashforth predictor*

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \frac{h}{24}(55\mathbf{y}'_k - 59\mathbf{y}'_{k-1} + 37\mathbf{y}'_{k-2} - 9\mathbf{y}'_{k-3})$$

and implicit fourth-order *Adams-Moulton corrector*

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \frac{h}{24}(9\mathbf{y}'_{k+1} + 19\mathbf{y}'_k - 5\mathbf{y}'_{k-1} + \mathbf{y}'_{k-2})$$

Adams Methods

Stability and accuracy of some multistep Adams methods

Explicit Methods		
Order	Stability threshold	Error constant
1	-2	1/2
2	-1	5/12
3	-6/11	3/8
4	-3/10	251/720

Implicit Methods		
Order	Stability threshold	Error constant
1	$-\infty$	-1/2
2	$-\infty$	-1/12
3	-6	-1/24
4	-3	-19/720

- ▶ *Stability threshold* indicates left endpoint of stability interval for scalar ODE
- ▶ *Error constant* indicates coefficient of h^{p+1} term in local truncation error, where p is order of method
- ▶ Implicit methods are both more stable and more accurate than corresponding explicit methods of same order

BDF Methods

- ▶ *Backward differentiation formulas* form another important family of implicit multistep methods
- ▶ BDF methods, typified by popular formula

$$\mathbf{y}_{k+1} = \frac{1}{11}(18\mathbf{y}_k - 9\mathbf{y}_{k-1} + 2\mathbf{y}_{k-2}) + \frac{6h}{11}\mathbf{y}'_{k+1}$$

are effective for solving stiff ODEs

⟨ interactive example ⟩

Implicit Multistep Methods

- ▶ Implicit methods have much larger stability region than explicit methods, but must be iterated to convergence to enjoy this benefit fully
 - ▶ PECE scheme is actually explicit, though in a somewhat complicated way
- ▶ Although implicit methods are more stable than explicit methods, they are still not necessarily unconditionally stable
 - ▶ No multistep method of greater than second order is unconditionally stable, even if it is implicit

Multistep Methods – Pros and Cons

Advantages of multistep methods

- ▶ Good local error estimate can be determined from difference between predictor and corrector
- ▶ Being based on interpolation, they can efficiently provide solution values at output points other than integration points
- ▶ Can be effective for stiff ODEs

Disadvantages of multistep methods

- ▶ Not self-starting, since several previous values of $\mathbf{y}_k$ are needed initially
- ▶ Changing step size is complicated, since interpolation formulas are most conveniently based on equally spaced intervals for several consecutive points
- ▶ Relatively complicated to program

Multivalued Methods

Multivalue Methods

- ▶ With multistep methods it is difficult to change step size, since past history of solution is most easily maintained at equally spaced intervals
- ▶ Like multistep methods, *multivalue methods* are also based on polynomial interpolation, but avoid some implementation difficulties associated with multistep methods
- ▶ One key idea motivating multivalue methods is observation that interpolating polynomial itself can be evaluated at any point, not just at equally spaced intervals
- ▶ Equal spacing associated with multistep methods is artifact of representation as linear combination of successive solution and derivative values with fixed weights

Multivalue Methods, continued

- ▶ Another key idea in implementing multivalue methods is representing interpolating polynomial so that parameters are values of solution and its derivatives at t_k
- ▶ This approach is analogous to using Taylor, rather than Lagrange, representation of polynomial
- ▶ Solution is advanced in time by simple transformation of representation from one point to next, and it is easy to change step size
- ▶ Multivalue methods are mathematically equivalent to multistep methods but are more convenient and flexible to implement, so most modern software implementations are based on them

Example: Multivariate Method

$$y(t+h) = y(t) + h y'(t) + \frac{h^2}{2} y''(t) + \frac{h^3}{6} y'''(t) + \dots$$

Handwritten annotations: y_{k+1} under $y(t+h)$, y_k under $y(t)$, y_k under $y'(t)$, y_k under $y''(t)$, and y_k under $y'''(t)$.

- Consider four-value method for solving scalar ODE

$$y' = f(t, y)$$

- Instead of representing interpolating polynomial by its value at four different points, we represent it by its value and first three derivatives at single point t_k

$$\mathbf{y}_k = \begin{bmatrix} y_k \\ hy'_k \\ (h^2/2)y''_k \\ (h^3/6)y'''_k \end{bmatrix} \in \mathbb{R}^4$$

Handwritten annotations: A green circle around the vector $\mathbf{y}_k$ and the term y_k in the first component. A double equals sign $==$ is written to the left of the vector.

Example: Multivalue Method

- By differentiating Taylor series

$$y(t_k + h) = y(t_k) + hy' + \frac{h^2}{2}y'' + \frac{h^3}{6}y''' + \dots$$

three times, we see that corresponding values at next point $t_{k+1} = t_k + h$ are given approximately by transformation

$$\hat{\mathbf{y}}_{k+1} = \mathbf{B} \mathbf{y}_k$$

where matrix $\mathbf{B}$ is given by

$$\mathbf{B} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{y}_{k+1} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} y_k \\ hy'_k \\ \frac{h^2}{2}y''_k \\ \frac{h^3}{6}y'''_k \end{bmatrix}$$

$$y_{k+1}' = y_k' + h y_k'' + \frac{h^2}{2} y_k''' + \dots$$

Example: Multivalue Method

- ▶ We have not yet used ODE, however, so we add correction term to foregoing prediction to obtain final value

$$\mathbf{y}_{k+1} = \hat{\mathbf{y}}_{k+1} + \alpha \mathbf{r}$$

where $\mathbf{r}$ is fixed 4-vector and

$$\alpha = h(f(t_{k+1}, y_{k+1}) - \hat{y}'_{k+1})$$

- ▶ For consistency, i.e., for ODE to be satisfied, we must have $r_2 = 1$
- ▶ Remaining three components of $\mathbf{r}$ can be chosen in various ways, resulting in different methods, analogous to different choices of parameters in multistep methods
- ▶ For example, four-value method with

$$\mathbf{r} = \begin{bmatrix} \frac{3}{8} & 1 & \frac{3}{4} & \frac{1}{6} \end{bmatrix}^T$$

is equivalent to implicit fourth-order Adams-Moulton method

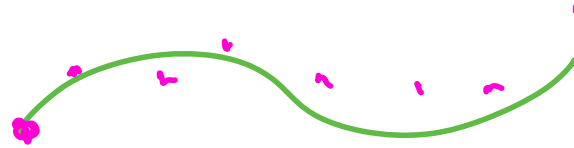
Multivalue Methods, continued

- ▶ It is easy to change step size with multivalue methods: merely rescale components of $\mathbf{y}_k$ to reflect new step size
- ▶ It is also easy to change order of method simply by changing components of $\mathbf{r}$
- ▶ These two capabilities, combined with sophisticated tests and strategies for deciding when to change order and step size, have led to development of powerful and efficient software packages for solving ODEs based on variable-order/variable-step methods

Variable-Order/Variable-Step ODE Solvers

- ▶ Analogous to adaptive quadrature routines
 - ▶ Automatically adapt to given problem
 - ▶ Vary order and step size to meet user-supplied error tolerance efficiently
- ▶ Avoid need for special starting procedure
 - ▶ Start with first-order method, which requires no additional starting values
 - ▶ Let automatic order/step size selection procedure increase order as needed to meet required accuracy
- ▶ Handle stiffness conveniently
 - ▶ Often have options for solving either stiff or nonstiff problems
 - ▶ Some detect stiffness automatically and select appropriate method

Summary – ODE Initial Value Problems



- ▶ Numerical solution of ODE IVP is table of approximate values of solution function at discrete points, generated by simulating behavior of system governed by ODE step by step
- ▶ Each step incurs *local* discretization error, as well as propagating previous errors, which combine to determine overall *global* error
- ▶ Whether errors are amplified or diminished over time depends on whether true solution is stable or unstable
- ▶ *Euler's method* advances solution by extrapolating along straight line whose slope is given by right-hand side of ODE
- ▶ Scalar test ODE $y' = \lambda y$ shows accuracy of Euler's method to be first-order and provides stability bound $h \leq -2/\lambda$ on step size for $\lambda < 0$

Summary – ODE Initial Value Problems

- ▶ Accuracy can be improved by using higher-order methods, and stability region can be expanded by using *implicit* methods
- ▶ Implicit *trapezoid method*, for example, is second-order accurate and unconditionally stable
- ▶ Implicit methods are especially important for solving *stiff* ODEs, which have widely disparate time scales, so that step size is more severely restricted by stability than by required accuracy
- ▶ Important families of ODE methods include *Runge-Kutta* and *multistep/multivalued* methods